

بخش Screen Manager — A4 و معماری Screen-Based

مقدمه

این سند مربوط به مرحله A4 از فاز اول پروژه است.

پس از تثبیت: A1: دسته‌بندی Event ها - A2: قرارداد داده‌ای A3 - Event Object: جریان Event در کل سیستم

در مرحله A4 تمرکز بر مدیریت حالت‌های مختلف رابط کاربری (Screenها) و جداسازی کامل آن‌ها از منطق بازی است.

این سند توضیح می‌دهد که: Screen چیست - ScreenManager چه نقشی دارد - چرا Screenها «اتاق‌های در بسته» هستند - و چگونه A4 بدون آسیب به A2 پیاده‌سازی شده است

هدف مرحله A4

هدف اصلی A4 پاسخ به این سؤال است:

«بازی در چه صفحه‌هایی اجرا می‌شود و چه کسی مسئول مدیریت و جابه‌جایی بین آن‌هاست؟»

پس از پایان A4: - هر فاز بازی UI مستقل خود را دارد - هیچ منطق بازی داخل UI نوشته نمی‌شود - main.py از مدیریت Screenها پاک می‌شود - تغییر Screenها به صورت Event-driven انجام می‌شود

مفهوم Screen در معماری پروژه

در این پروژه، Screen به معنای یک حالت فعال از بازی است که: - UI مخصوص به خود دارد - Input مخصوص به خود را می‌پذیرد - فقط مسئول نمایش و ارسال Event است

Screenها منطق بازی ندارند و تصمیم‌گیری انجام نمی‌دهند.

Screen به عنوان «اتاق در بسته»

در معماری Screen-Based، هر Screen مانند یک اتاق در بسته در نظر گرفته می‌شود:

- آنچه داخل Screen است فقط به همان Screen تعلق دارد
- هیچ UI یا Input از Screen دیگر قرض گرفته نمی‌شود
- Screenها فقط Model و EventBus مشترک دارند

مثال مفهومی:

- RecruitScreen: اتاق خرید (تعامل دارد)
- CombatScreen: اتاق نبرد (فقط نمایش)

ساختار پوشه UI

```
src/ui/
├── screen_manager.py
├── services.py
├── screens/
│   ├── base_screen.py
│   ├── recruit_screen.py
│   └── battle_screen.py
└── components/
    └── card_view.py
```

BaseScreen (قرارداد Screenها)

فایل `base_screen.py` یک قرارداد رسمی تعریف می‌کند که تمام Screenها باید از آن پیروی کنند.

متدهای اجباری:

- `handle_event(event)`
- `update(dt)`
- `render(surface)`

این قرارداد تضمین می‌کند که ScreenManager بتواند با تمام Screenها به صورت یکنواخت کار کند.

ScreenManager

نقش ScreenManager

ScreenManager مسئول: - نگهداری Screen فعال - مدیریت تغییر Screenها - ارسال Eventها به Screen فعال

ScreenManager هیچ منطق بازی ندارد و فقط مدیریت جریان UI را انجام می‌دهد.

تغییر Screen به صورت Event-Driven

در این پروژه، تغییر Screen مستقیماً انجام نمی‌شود، بلکه از طریق Event سیستمی انجام می‌شود:

```
{
  "type": "SYS_PHASE_CHANGED",
  "source": "ui_manager",
  "target": null,
  "payload": {"new_phase": "COMBAT"}
}
```

این طراحی باعث می‌شود: - Screen switching با A2 سازگار بماند - در فاز Server نیز بدون تغییر قابل استفاده باشد

RecruitScreen

ویژگی‌ها:

- تنها Screen دارای Input کاربر
- ارسال CMD Event ها (مثلاً Damage Test)
- ساخت CardView مخصوص خودش

محدودیت‌ها:

- هیچ منطق بازی ندارد
- فقط Event استاندارد می‌سازد و ارسال می‌کند

CombatScreen

ویژگی‌ها:

- Screen نمایشی (View-only)
- بدون Input کاربر
- بدون CardView یا Model مستقیم

هدف:

آماده‌سازی برای نمایش EVENT های Combat در مراحل بعدی

services.py

فایل `services.py` شامل ابزارهای کمکی UI است، مانند: - بارگذاری تصاویر - مدیریت فونت‌ها - رسم متن

این فایل: - Screen نیست - Logic ندارد - در A4 تغییری نمی‌کند

ارتباط A4 با A2

مرحله A4 بدون هیچ تغییری در قرارداد A2 پیاده‌سازی شده است: - تمام Eventها همچنان دارای `source` , `type` , `target` , `payload` هستند - Screenها فقط مصرف‌کننده Eventها هستند - هیچ Event خارج از قرارداد ساخته نمی‌شود

خروجی نهایی مرحله A4

مرحله A4 زمانی کامل است که: - ScreenManager وجود داشته باشد - BaseScreen به‌عنوان قرارداد تعریف شده باشد - Combat Screen و Recruit از هم جدا باشند - تغییر Screen به‌صورت Event-driven انجام شود - A2 بدون نقض باقی مانده باشد

تمام این موارد در پیاده‌سازی فعلی رعایت شده است.

جمع‌بندی

مرحله A4 پروژه را از یک برنامه تک‌صفحه‌ای به یک بازی چندحالتی با معماری تمیز تبدیل می‌کند.

این مرحله پایه‌ای حیاتی برای: - Combat Visualization - Recruit Phase Logic - اتصال به Server

است.

پایان سند مرحله A4